Métodos Cuantitativos

Clase 10 - Regresión lineal parte 3

Unidad II: Test de Hipótesis y Regresión Lineal

¿Qué veremos hoy?

- **Repaso de evaluación del modelo**
- Extrapolación y predicciones
- Regresión no lineal
- Overfitting y entrenamiento-prueba

Conceptos claves

- **Variable dependiente**
  - Es la variable que queremos predecir o explicar.
- **Variable(s) independiente(s)**
  - Son las variables que utilizamos para predecir la variable dependiente.
- **Intercepto**
  - Es el valor de y cuando x es cero.
- **Pendiente**
  - Representa el cambio en y por cada unidad de cambio en x.
- **Error:**

Breve repaso: Supuestos

- En la clase anterior establecimos que, para contar con un modelo de Regresión Lineal **robusto**, es imprescindible satisfacer los supuestos de **linealidad**, **normalidad**, **homocedasticidad** e **independencia**.
- Incumplir estos supuestos puede comprometer la validez de nuestro modelo y provocar que las interpretaciones realizadas sobre el modelo y sus coeficientes sean erróneas.

Breve repaso: Supuestos

1. **Linealidad**: La relación entre las variables independientes y la variable dependiente es lineal.
2. **Normalidad**: Los errores de la regresión siguen una distribución normal.
3. **Homocedasticidad**: La varianza de los errores es constante en todos los niveles de las variables predictoras.
4. **Independencia**: Los errores de la regresión no están correlacionados entre sí.

Breve repaso: Coeficientes

- Los coeficientes **representan la relación entre la variable dependiente y cada una de las variables independientes.**
- La interpretación de estos coeficientes es **crucial para entender el impacto que tiene cada variable independiente en la variable dependiente.**

Breve repaso: Coeficientes

Breve repaso: Coeficientes

```python
# seleccionamos las variables independientes
X = data[["edad", "imc", "fumador", "mujer"]]

# seleccionamos las variables dependientes
y = data["gastos_seguro_medico"]

# agregamos el intercepto a los datos
X = sm.add_constant(X)

# ajustamos el modelo
modelo = sm.OLS(y, X)

# generamos los resultados
resultado = modelo.fit()
```

Breve repaso: Coeficientes

```
# obtenemos los coeficientes
resultado.params
```

```
const     -11287.619146
edad         258.313477
imc          311.151783
fumador    23964.879376
mujer         35.885956
dtype: float64
```

- β_0 = -11287.619146
- β_1 = 258.313477
- β_2 = 311.151783
- β_3 = 23964.879376
- β_4 = 35.885956

Breve repaso: Coeficientes

- $\beta_0 = -11287; \beta_1 = 258; \beta_2 = 311; \beta_3 = 23964; \beta_4 = 35$
- $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4 + \varepsilon_i$

	edad	imc	fumador	mujer	gastos_seguro_medico	prediccion
0	19	27.90	1	1	16884.9240	26302.237002
1	18	33.77	0	0	1725.5523	3869.619160

- $y = -11287 + 258 \cdot 19 + 311 \cdot 27.90 + 23964 \cdot 1 + 35 \cdot 1$
- $y \approx 26302.2$
- $y = -11287 + 258 \cdot 18 + 311 \cdot 33.77 + 23964 \cdot 0 + 35 \cdot 0$
- $y \approx 3869.6$

Breve repaso: Evaluación del modelo

- Para evaluar nuestro modelo hemos usado una serie de métricas de rendimiento, las cuales son ampliamente utilizadas en la evaluación de modelos de regresión.
- Estas métricas nos ayudan a **cuantificar la calidad de nuestras predicciones** y a **comparar diferentes modelos entre sí**.

Evaluación del modelo: R^2

- El **coeficiente de determinación** R^2, nos indica la proporción de la variabilidad en la variable dependiente que es explicada por nuestro modelo.
  - Un R^2 igual a 0 indica que el modelo no está capturando ninguna información útil sobre los datos.
  - Un R^2 igual a 1 indica un ajuste perfecto del modelo a los datos. Esto significa que todas las variaciones en la variable de respuesta son explicadas por el modelo.

Evaluación del modelo: R^2

```
X = data[["edad", "imc"]]
y = data["gastos_seguro_medico"]
X = sm.add_constant(X)

modelo = sm.OLS(y, X)
resultado = modelo.fit()

# reportamos el R^2
print(resultado.rsquared)
```

```
0.11711811815482553
```

- En este caso, el R^2 es 0.1171, lo que significa que usando solo la **edad y el IMC** podemos explicar aproximadamente el 11.71% de las **razones por las que las personas gastan lo que gastan en su seguro médico**.

Evaluación del modelo: R^2

```
X = data[["edad", "imc", "fumador", "mujer"]]
y = data["gastos_seguro_medico"]
X = sm.add_constant(X)

modelo = sm.OLS(y, X)
resultado = modelo.fit()

# reportamos el R^2
print(resultado.rsquared)
```

```
0.7470114569681708
```

- Al agregar las variables independientes `fumador` y `mujer`, el R^2 aumenta a 0.7470, lo que significa que al usando las variables **edad, IMC, fumador y mujer** podemos explicar aproximadamente el 74.70% de las **razones por las que las personas gastan lo que gastan en su seguro médico**.

Evaluación del modelo: R^2

```
X = data[["edad", "imc", "fumador", "mujer"]]
y = data["gastos_seguro_medico"]

modelo = sm.OLS(y, X)
resultado = modelo.fit()

# reportamos el R^2
print(resultado.rsquared)
```

```
0.872505325660148
```

- Al eliminar el intercepto del modelo, el R^2 aumenta a 0.8725, lo que significa que al usando las variables **edad, IMC, fumador y mujer** podemos explicar aproximadamente el 87.25% de las **razones por las que las personas gastan lo que gastan en su seguro médico**.

Evaluación del modelo: R^2

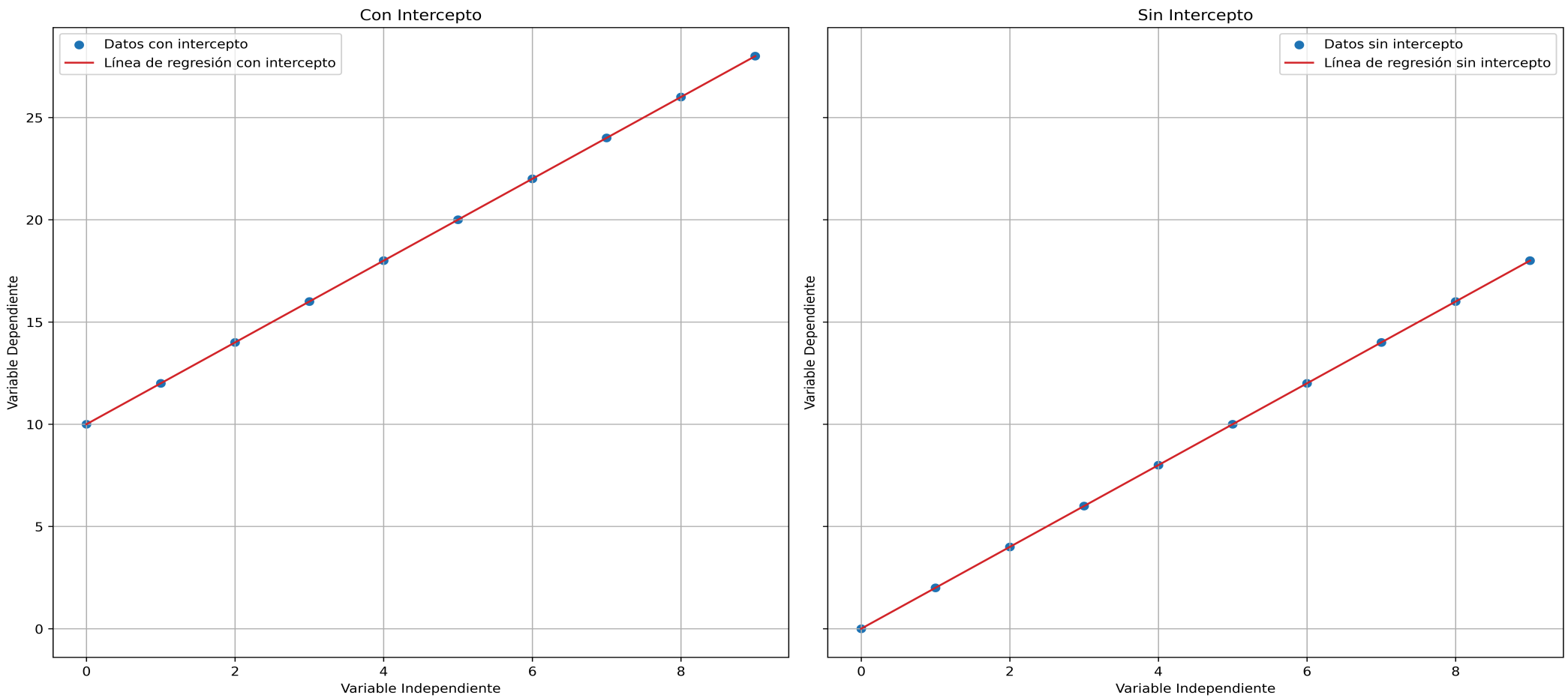
- El R^2 no siempre es una métrica confiable porque sólo mide cuánta variabilidad de los datos puede ser explicada por el modelo, sin considerar si el modelo es realmente apropiado o si las predicciones son precisas.

Comprobación de Linealidad

```
residuos = resultado.resid
round(np.mean(residuos), 4)
```

```
np.float64(-345.2709)
```

Digresión: ¿Con o sin intercepto?



Solo se debe omitir el intercepto cuando existe una justificación clara basada en el conocimiento del dominio y la

naturaleza de los datos.

Breve repaso: MSE

- Una forma de cuantificar los errores del mediante el **Error Cuadrático Medio** (MSE, por sus siglas en inglés). Esta medida se expresa de la siguiente manera:
- $\mathrm{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y_i})^2$
- El MSE calcula la diferencia cuadrática promedio entre los valores observados y_i y los valores predichos $\hat{y_i}$.

Breve repaso: RMSE

- El RMSE es la raíz cuadrada del MSE ($\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$), y la principal diferencia es que los errores en las mismas unidades que la variable de respuesta.
- Un RMSE más bajo indica que las predicciones del modelo están más cerca de los valores reales, lo que sugiere un mejor ajuste del modelo a los datos.

Evaluación del modelo: RMSE

```python
from statsmodels.tools.eval_measures import rmse
X = data[["edad", "imc", "fumador", "mujer"]]
y = data["gastos_seguro_medico"]
X = sm.add_constant(X)

modelo = sm.OLS(y, X)
resultado = modelo.fit()

y_hat = resultado.fittedvalues
print(f"RMSE: {rmse(y, y_hat)}")
```

```
RMSE: 6110.990099443555
```

- El resultado significa que, en promedio, **las predicciones del modelo difieren de los valores reales en aproximadamente 6.111 unidades en la escala de la variable de respuesta** (en este caso, el gasto en seguro médico).

Contenidos clase

- **Extrapolación**
- Regresión no lineal
- Overfitting

¿Qué veremos hoy?

- Repaso de evaluación del modelo
- **Extrapolación y predicciones**
- Regresión no lineal
- Overfitting y entrenamiento-prueba

Extrapolación

Extrapolación

- La extrapolación en regresión lineal se refiere al proceso de utilizar el modelo de regresión para hacer predicciones fuera del rango de los datos con los que se entrenó el modelo.
- Es decir, se hace una estimación más allá de los valores máximos y mínimos observados de las variables independientes.

Extrapolación: Riesgos

- **Incertidumbre incrementada**: Los valores predichos fuera del rango original pueden ser menos precisos y confiables, ya que el modelo no fue validado en esos puntos.
- **Suposiciones no verificadas**: La relación lineal asumida por el modelo puede no sostenerse fuera del rango observado, llevando a predicciones erróneas.

Predicciones en statsmodels

Para este ejemplo usaremos los datos de la semana anterior, el cual analizaba el índice de rendimiento de un alumno en base a distintas variables.

```
   notas_previas  horas_estudiadas  indice_rendimiento
0             99                 7                  91
1             82                 4                  65
2             51                 8                  45
-----------------------------------------------------------
       notas_previas  horas_estudiadas  indice_rendimiento
count   10000.000000      10000.000000        10000.000000
mean       69.445700          4.992900           55.224800
std        17.343152          2.589309           19.212558
min        40.000000          1.000000           10.000000
25%        54.000000          3.000000           40.000000
50%        69.000000          5.000000           55.000000
75%        85.000000          7.000000           71.000000
max        99.000000          9.000000          100.000000
```

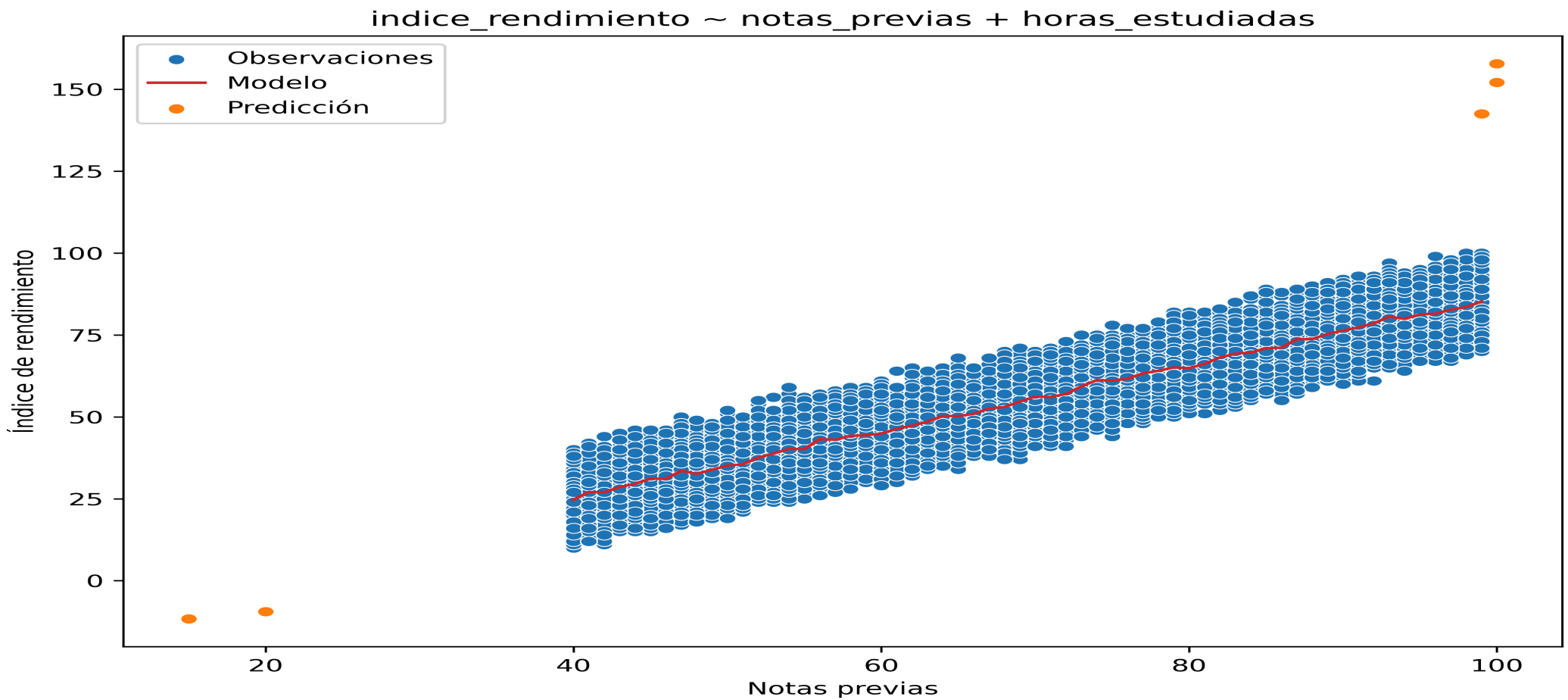
Predicciones en statsmodels

Para realizar predicciones con `statsmodels` el proceso es bastante simple:

```python
# creamos un DataFrame con los registros que buscamos predecir
registro = pd.DataFrame({
    "intercept": [1, 1, 1, 1, 1],
    "notas_previas": [100, 99, 100, 20, 15],
    "horas_estudiadas": [30, 25, 28, 0, 1]
})

# usamos el resultado del modelo para predecir
resultado.predict(registro)
```

```
0    157.824603
1    142.517294
2    152.109328
3     -9.434335
4    -11.672311
dtype: float64
```

Predicciones en statsmodels



Contenidos clase

- Extrapolación
- **Regresión no lineal**
- Overfitting

¿Qué veremos hoy?

- Repaso de evaluación del modelo
- Extrapolación y predicciones
- **Regresión no lineal**
- Overfitting y entrenamiento-prueba

Regresión no lineal

Regresión no lineal

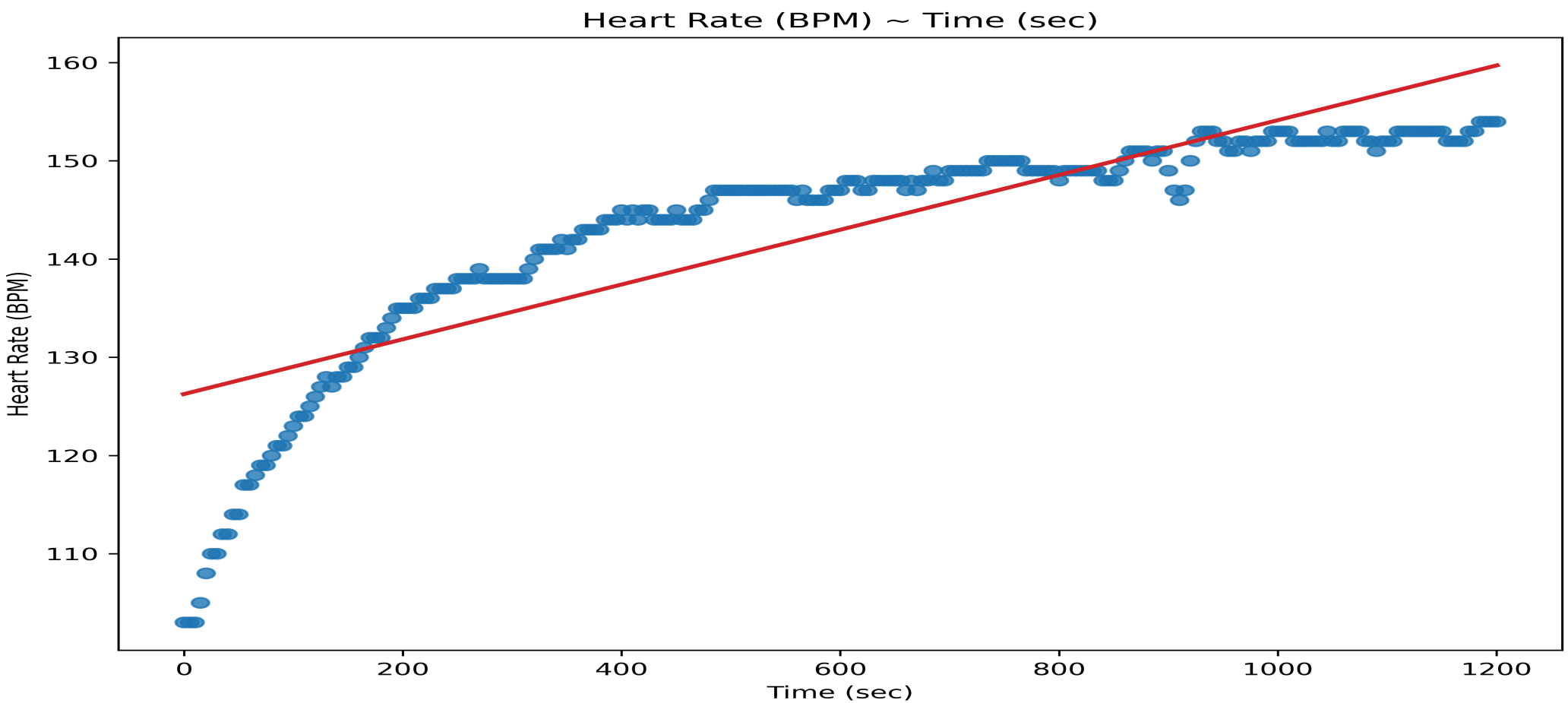
- Hasta el momento hemos estudiado que la regresión lineal es una herramienta estadística que supone la existencia de una relación lineal entre las variables independientes y la variable dependiente.
- Sin embargo, en la vida real, muchos fenómenos que estudiamos pueden exhibir relaciones más complejas que no se ajustan adecuadamente a un modelo lineal.
- Es en este contexto donde la **regresión no lineal** entra en juego, ofreciendo una mayor flexibilidad para modelar relaciones más complejas entre las variables.

Regresión no lineal

- A diferencia de la regresión lineal, **la regresión no lineal permite que la relación entre las variables independientes y la variable dependiente sea una función curva o de orden superior**.
- Lo que es crucial para captar dinámicas como las fluctuaciones cíclicas, crecimiento exponencial, o efectos de saturación que son comunes en muchos campos, desde la biología hasta la economía.

Regresión no lineal

Observemos los siguientes datos:



Regresión no lineal

- Una forma de ajustar este tipo de modelos, es usando la función `polyfit` de `numpy`, la cual nos permite ajustar un polinomio de un grado especifico a un conjunto de datos.
- Esta función es especialmente útil en la **aproximación de funciones** cuando se tiene una serie de puntos y se desea encontrar la mejor curva polinómica que los represente.

Regresión no lineal

```python
# definir las variables independientes
X = data[["Time (sec)"]].to_numpy().flatten()

# definir la variable dependiente
y = data["Heart Rate (BPM)"]

# definir el grado del polinomio
grado = 3

# ajustar los datos según el grado
coeficientes = np.polyfit(X, y, grado)

# obtener los polinomios ajustados
polinomio_ajustado = np.poly1d(coeficientes)

# obtener las predicciones
y_hat = polinomio_ajustado(X)
```

Regresión no lineal

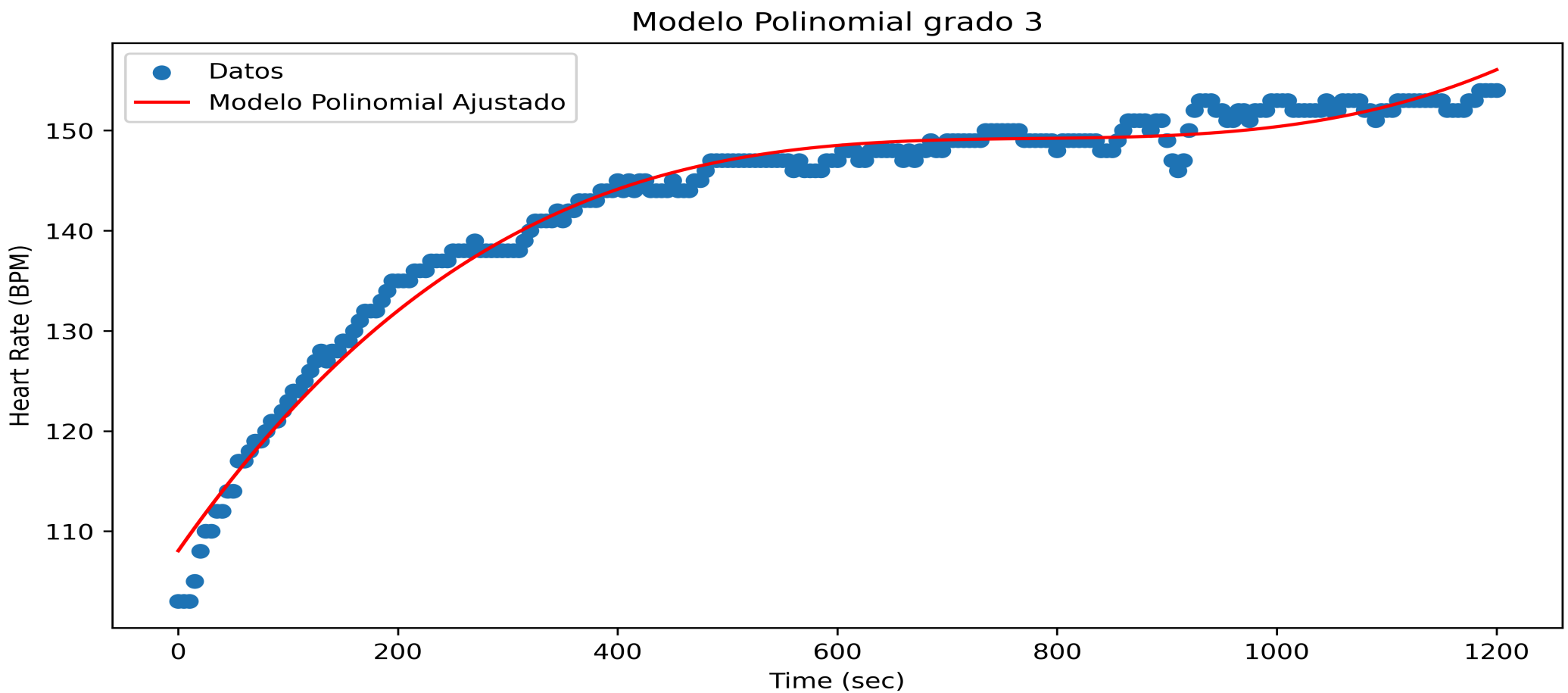
```
print(coeficientes)
```

```
[ 8.52749447e-08 -1.99196308e-04  1.56250847e-01  1.08054047e+02]
```

$$y = 8.52 \times 10^{-8} x^3 - 1.99 \times 10^{-4} x^2 + 0.15x + 108.05$$

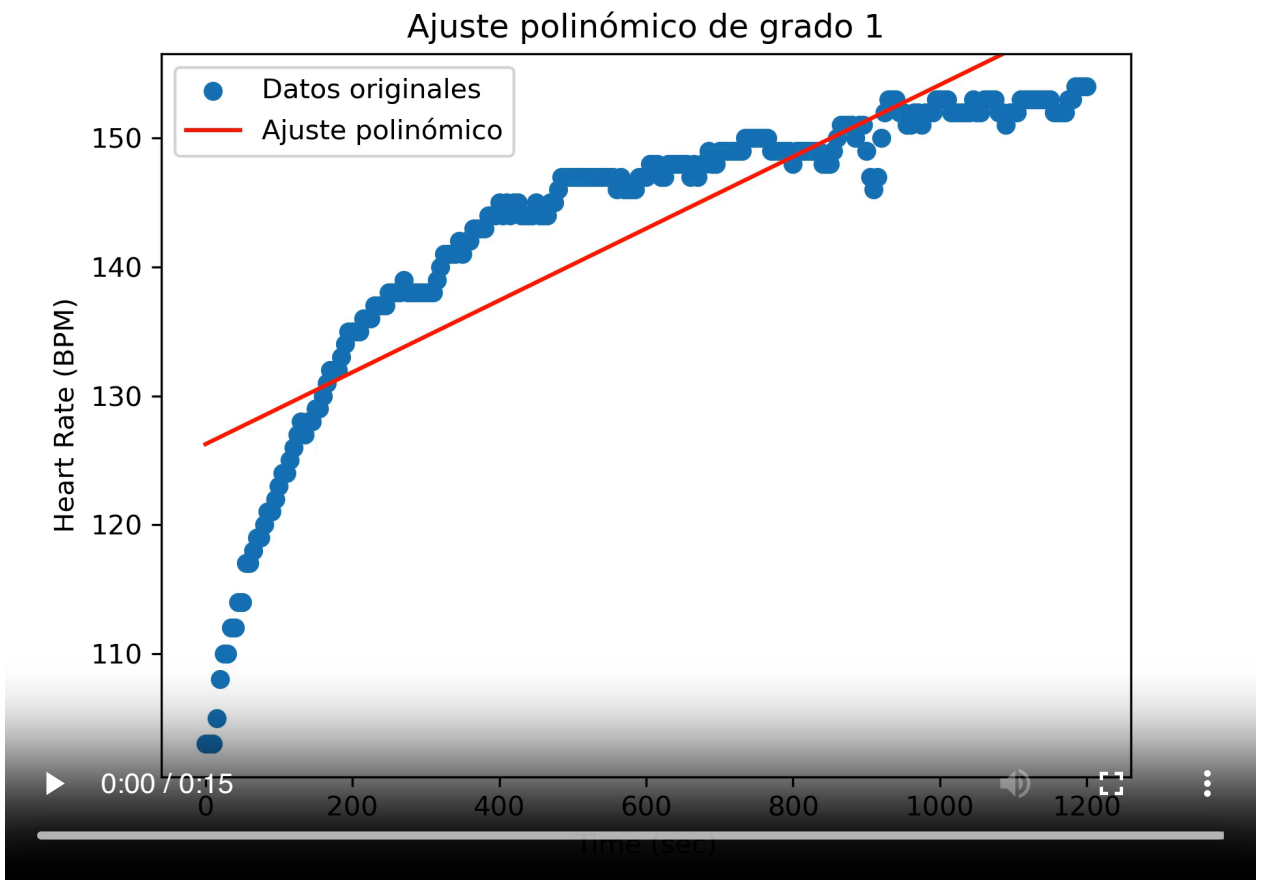
- En este caso, este tipo de regresiones son conocidas como **Regresión Polinómica**, con la forma:
  - $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_n x^n + \varepsilon.$
- Por lo que podríamos interpretar los coeficientes como:
  - $\beta_0 = 108.05$
  - $\beta_1 = 0.15$
  - $\beta_2 = -1.99 \times 10^{-4}$
  - $\beta_3 = 8.52 \times 10^{-8}$

Regresión no lineal



¿Qué sucede si aumentamos el grado del polinomio?

Regresión no lineal



Regresión no lineal

- En base a lo observado anteriormente, es válido cuestionarse: ¿por qué no ajustamos un **polinomio de grado muy alto para que capte lo mejor posible nuestros datos**?
- El principal problema de esta solución radica en un fenómeno conocido como *overfitting* o sobreajuste.

Contenidos clase

- Extrapolación
- Regresión no lineal
- **Overfitting**

¿Qué veremos hoy?

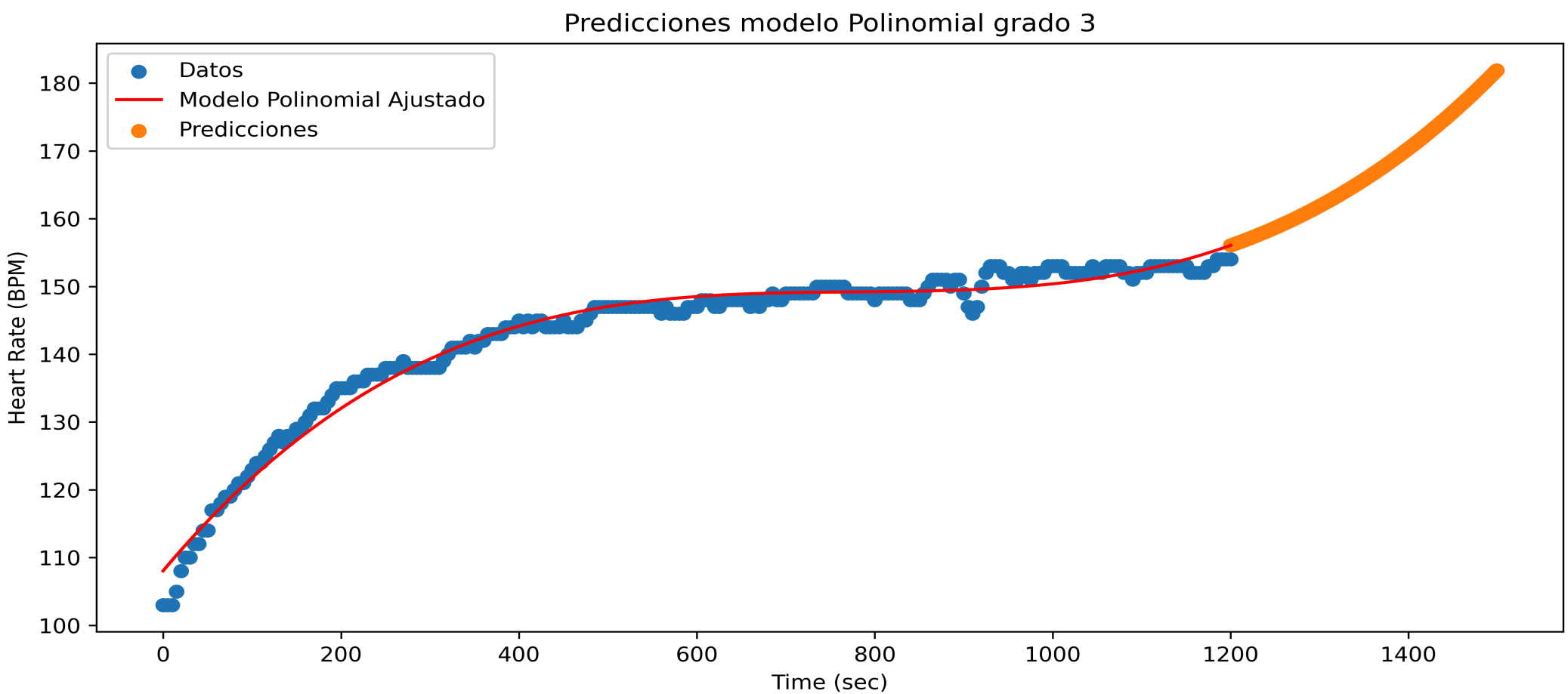
- Repaso de evaluación del modelo
- Extrapolación y predicciones
- Regresión no lineal
- **Overfitting y entrenamiento-prueba**

Overfitting

Overfitting

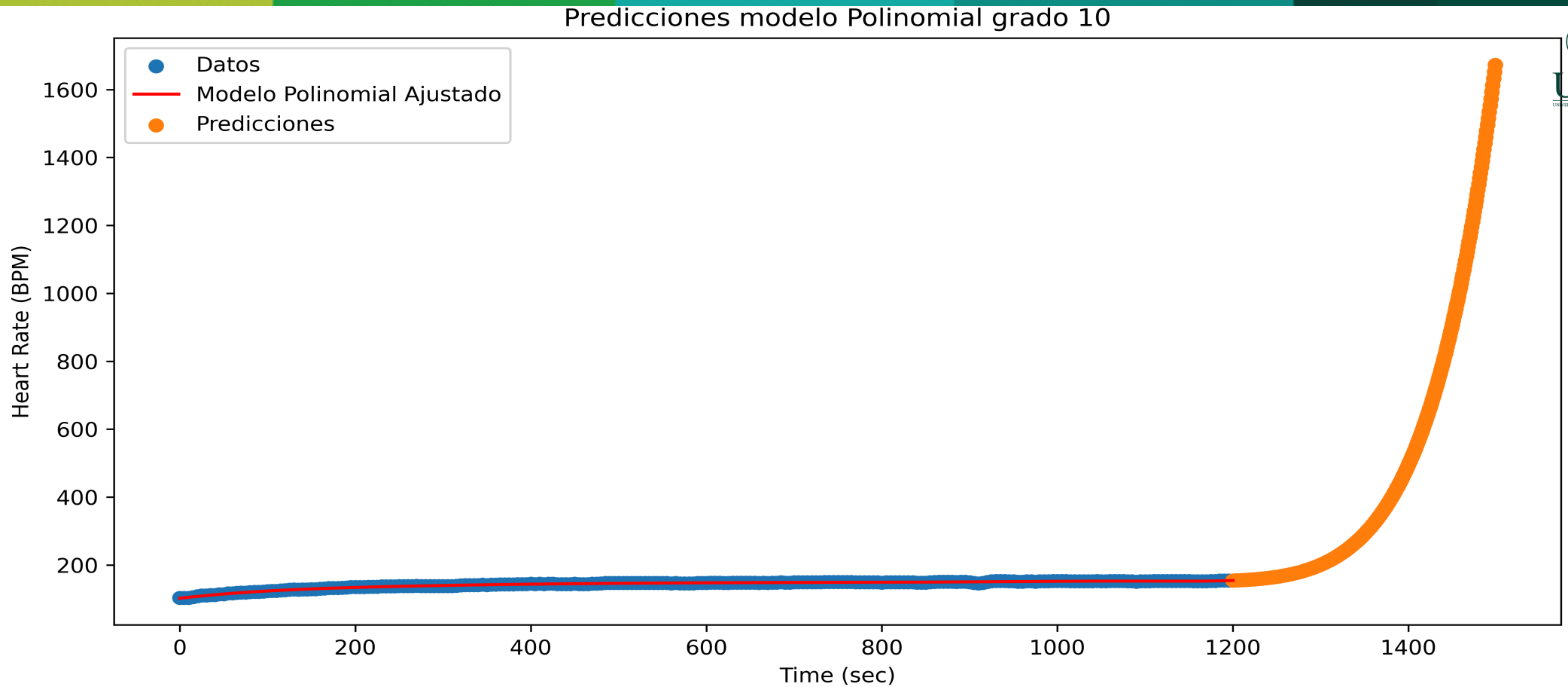
Overfitting

Observemos como el modelo polinomial de grado 3 predice las pulsaciones cardíacas para el tiempo que no está medido.



Overfitting

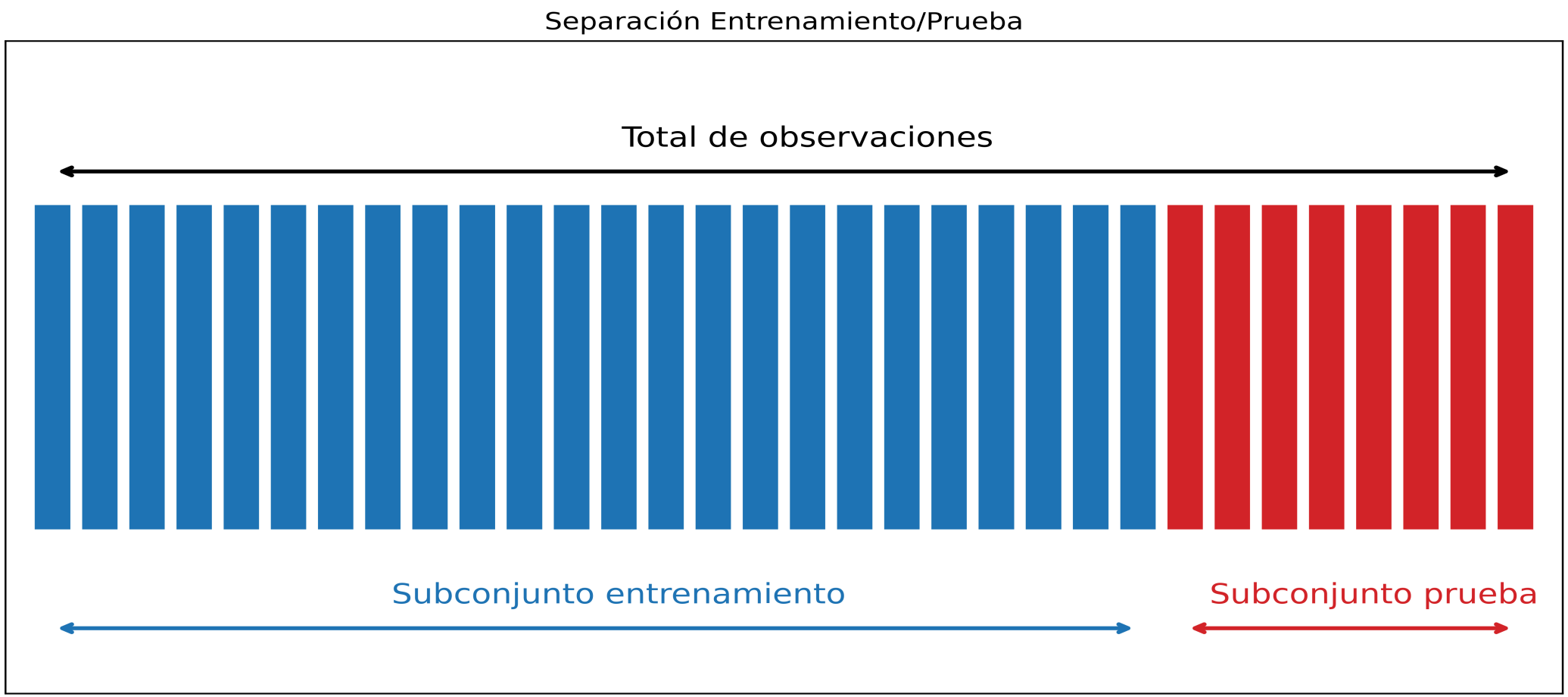
Y ahora observemos cómo el modelo polinomial de grado 10 predice las pulsaciones cardíacas para el tiempo que no está medido.


Predicciones modelo Polinomial grado 10
Datos
Modelo Polinomial Ajustado
Predicciones
Heart Rate (BPM)
Time (sec)
1600
1400
1200
1000
800
600
400
200
0
200
400
600
800
1000
1200
1400

Overfitting

- Una de las técnicas más comunes en ciencia de datos para evitar el problema del overfitting es **dividir los datos en dos subconjuntos**:
  - Entrenamiento (el subconjunto con el cual ajustamos el modelo)
  - Prueba
- El propósito de esta separación es **ajustar el modelo utilizando los datos de entrenamiento** y luego **validar la calidad del ajuste mediante una métrica de rendimiento (como el RMSE) con los datos de prueba**.

Separar datos Entrenamiento-Prueba



Separar datos Entrenamiento-Prueba

Separar datos Entrenamiento-Prueba

```python
from sklearn.model_selection import train_test_split

# importamos datos -estos son sólo de ejemplo-
url = "https://apmonitor.com/che263/uploads/Main/heart_rate.txt"
data = pd.read_csv(url)

# separamos las variables independientes y dependiente
X = data[["Time (sec)"]]
y = data["Heart Rate (BPM)"]

# creamos los subconjuntos, fijamos el test_size en 0.3
X_entrenamiento, X_prueba, y_entrenamiento, y_prueba = train_test_split(
  X, y, test_size=0.3, random_state=42)

print(f"Total de datos: {X.shape}")
print(f"Datos entrenamiento: {X_entrenamiento.shape}")
print(f"Datos prueba: {X_prueba.shape}")
```

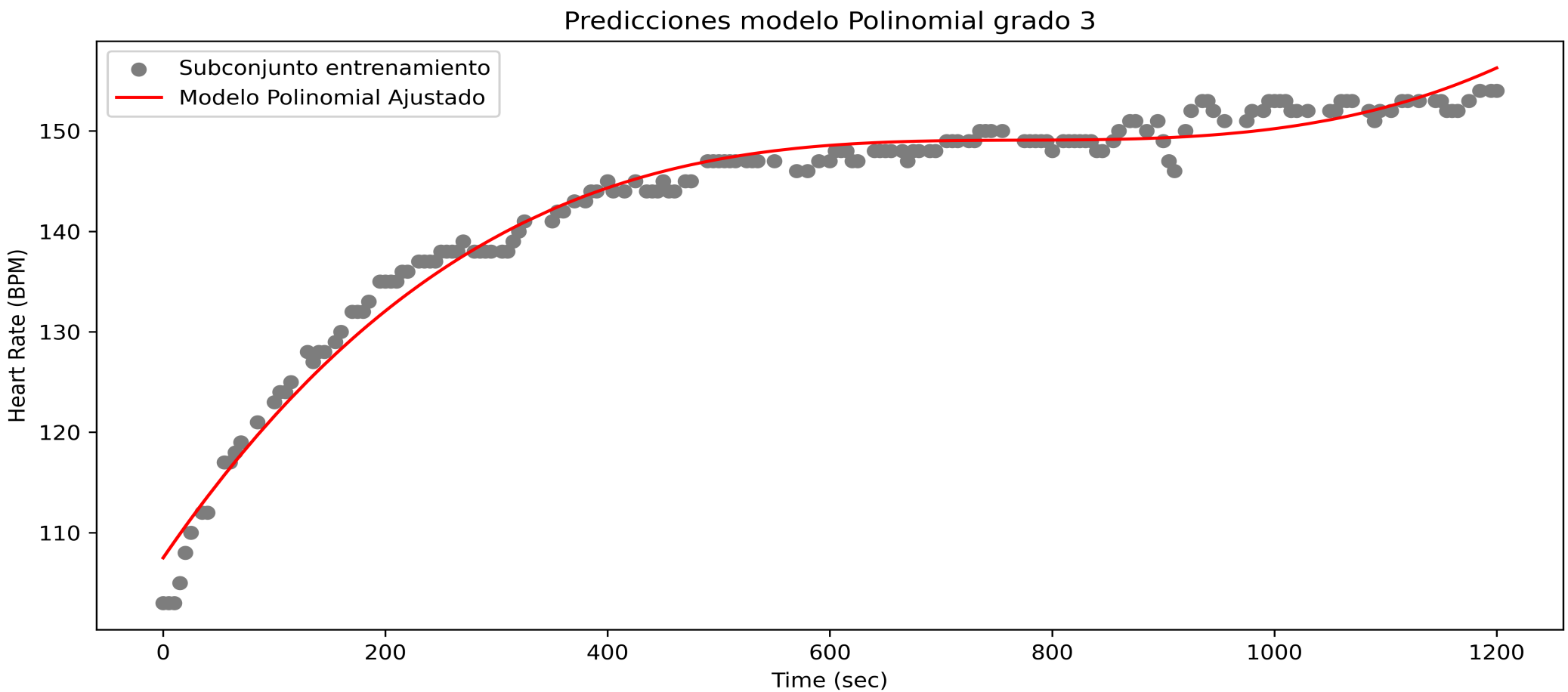
```
Total de datos: (241, 1)
Datos entrenamiento: (168, 1)
Datos prueba: (73, 1)
```

Separar datos Entrenamiento-Prueba

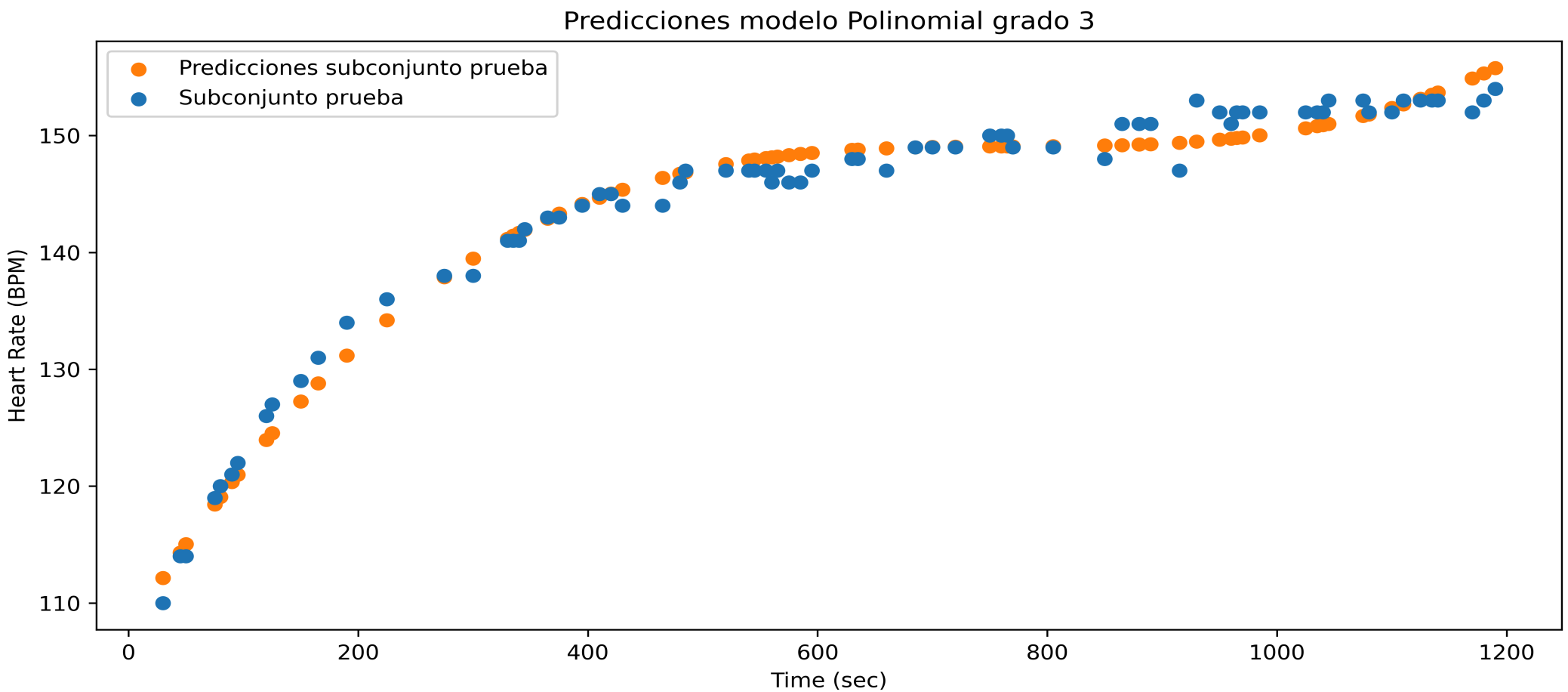
```python
from statsmodels.tools.eval_measures import rmse

# arreglamos los datos para el formato de polyfit
X_entrenamiento = X_entrenamiento.to_numpy().flatten()
X_prueba = X_prueba.to_numpy().flatten()

# ajustamos un modelo
grado = 3
coeficientes = np.polyfit(X_entrenamiento, y_entrenamiento, grado)
polinomio_ajustado = np.poly1d(coeficientes)
y_hat = polinomio_ajustado(X_prueba)
print(f"RMSE Subconjunto prueba: {rmse(y_prueba, y_hat)}")
```

```
RMSE Subconjunto prueba: 1.465952163552957
```

Separar datos Entrenamiento-Prueba



Separar datos Entrenamiento-Prueba



Bibliografía recomendada

- Estadística, Capítulo 10, Triola Mario F, 2018 12a edición.

Actividad práctica

- A continuación, para poner en práctica todos los conceptos aprendidos, realizaremos una actividad guiada en Colab.